### *Continuous data ingestion using Auto Loader with Structured Streaming*

The code in this section uses Auto Loader plus Structured Streaming to format the incoming `JSON` files as `Delta`. The table name we use is `synthetic_transactions`. Before processing data, we create a widget, see *Figure 3.5*, to determine if we want to reset the schema and checkpoint history. Resetting the history can be helpful when altering or debugging pipelines. If you reset (remove) your checkpoint history, you will reprocess all historical data.

```
dbutils.widgets.dropdown(name='Reset', defaultValue='False', choices=['True', 'False'], label="Reset Checkpoint and Schema")
```

Figure 3.5 – Creating a widget to reset the checkpoint and schema

Next, we set our variables for the notebook, mainly paths:

```
table_name = "synthetic_transactions"
raw_data_location = f"{volume_file_path}/schema_change_data/"
schema_location = f"{volume_file_path}/{table_name}/schema"
checkpoint_location = f"{volume_file_path}/{table_name}/checkpoint"
```

Figure 3.6 – Setting path variables

The setup file provides us with the `volume_file_path`, where we store the transaction data, schema, and checkpoint folders.

As shown in *Figure 3.7*, we add spark configurations to optimize and reduce the sample size for schema inference:

```
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", True)
spark.conf.set("spark.databricks.delta.autoCompact.enabled", True)
spark.conf.set("spark.databricks.cloudFiles.schemaInference.sampleSize.numFiles",1)
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", True)
```

Figure 3.7 – Setting configurations to optimize and reduce sample size

You can set these Spark configurations in the cluster's advanced options. These configurations will automatically compact sets of small files into larger files for optimal read performance. The streaming command is composed of a `readStream` and a `writeStream`. Let's walk through each line of code:

1. Starting with the readStream, the incoming stream format is `CloudFiles`. This means the files located in cloud storage. In this case, we generate data and write it to cloud storage. However, creating a stream with `.format("kafka")` is also possible. In that case, the stream would ingest directly from the data stream without writing to cloud storage first. Back to our project, we designate the file format as JSON.